PROJET JAVA

LE BONHOMME ET LE CHASSEUR

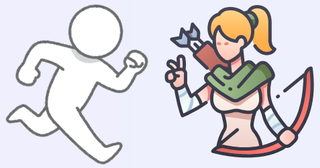
Sommaire

I. Le diagramme de classe
II. Le diagramme de séquence
III. Le jeu minimaliste
IV. Le Patron de Conception Adaptateur
V. Le Patron de Conception Stratégie
VI. L'Observer
VII. Nos difficultés
VIII. Pour aller plus loin

Suivant

NOTRE JEU

2 JOUEURS



GRILLE COMPOSÉ DE CELLULES

AVEC DES ELEMENTS FRUIT & FEU

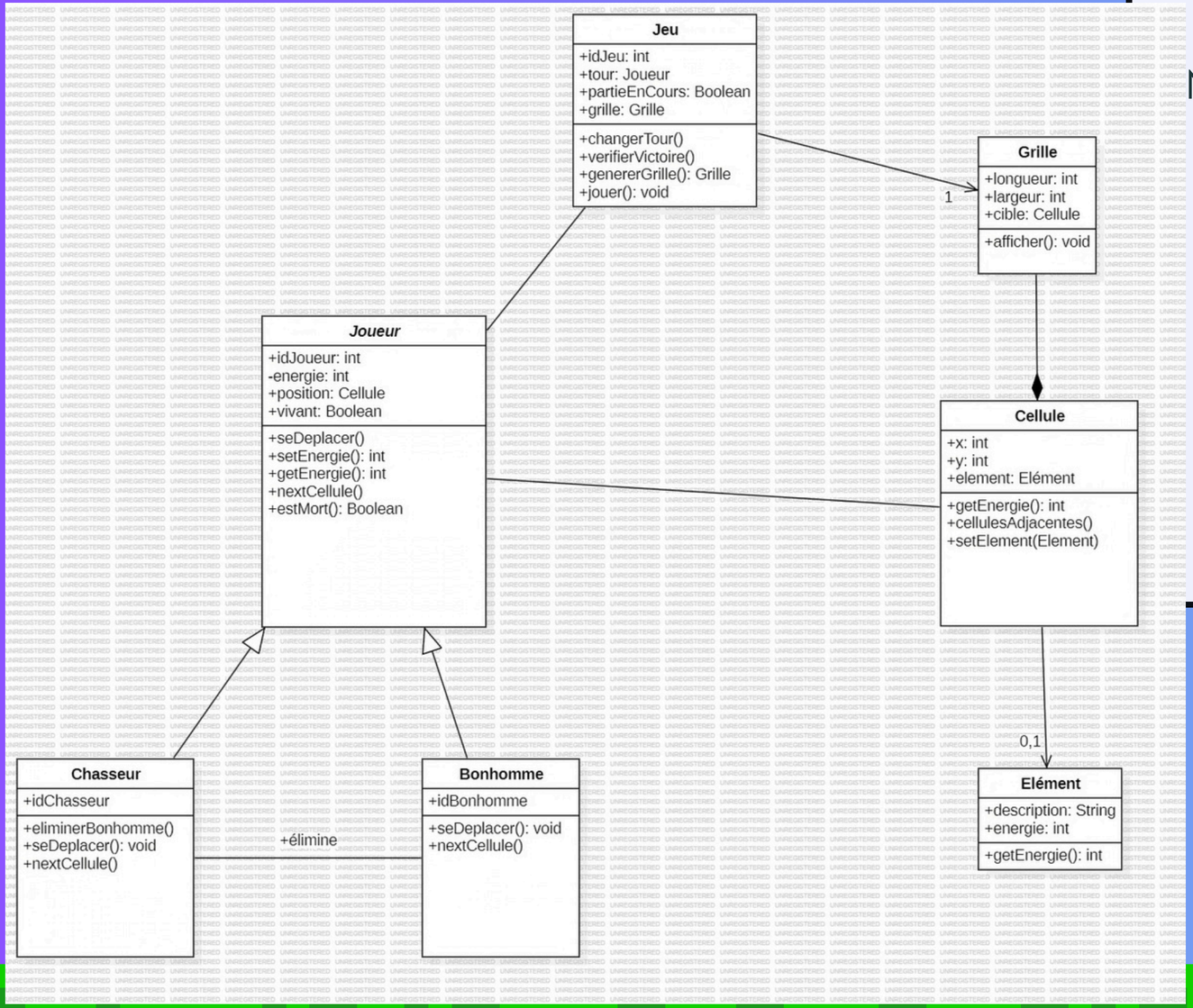
CHAQUE DEPLACEMENT COUTE 1 ENERGIE AU JOUEUR

FIN DU JEU:

- UN JOUEUR ATTEINT LA CIBLE
- CHASSEUR ATTRAPE JOUEUR
- JOUEURS N'ONT PLUS D'ENERGIE

Suivant

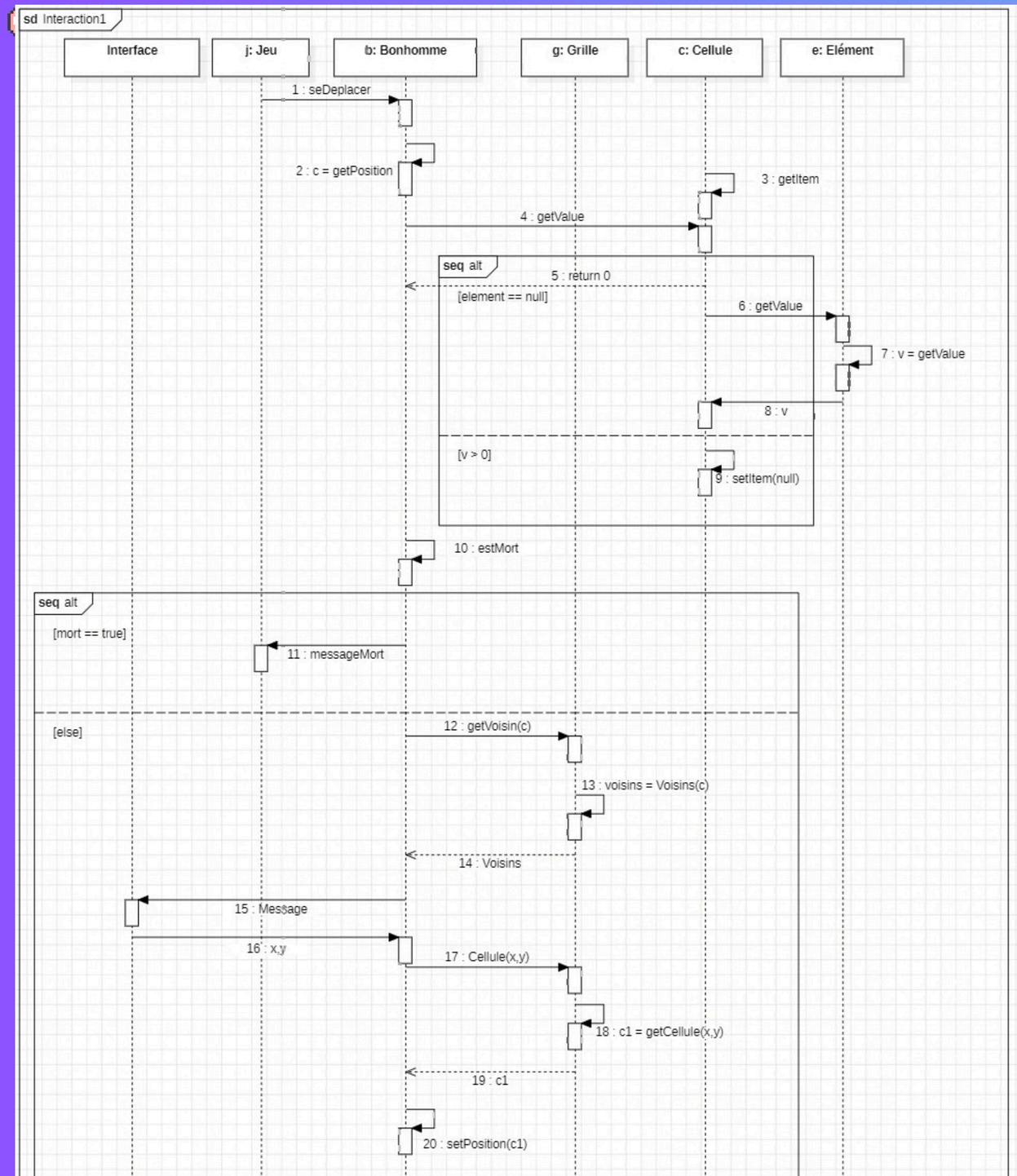
LE DIAGRAMME DE CLASSE



NOS CLASSES IMPORTANTES:

- GRILLE
- CELLULE (contient au max 1 élément)
- ELEMENT (modifie l'énergie des joueurs)
- JOUEUR (se déplace de cellule en cellule)
- BONHOMME (connait la position de la cible et s'oriente vers)
- CHASSEUR (se deplacer vers la cible et le joueur)

Suivant



LE DIAGRAMME DE SEQUENCE

- Méthode se déplacer
- nous a permis de structurer notre jeu et notre manière de concevoir l'intéraction entre objets

Suivant

NOTRE VERSION MINIMALE

```java
public Grille(int n) {
    this.n = n;
    this.grille = new Cellule[n][n];
    initialiser();
}
```

Plutôt que d'avoir une simple matrice de chiffres ou de caractères, notre Grille est un agrégat d'objets Cellule. Chaque cellule est instanciée avec ses propres coordonnées (i,j) dès le départ.

```java
public void initialiser() {
    for(int i=0; i<n; i++) {
        for (int j=0; j<n; j++) {
            grille[i][j]= new Cellule(i,j);
        }
    }
    placerCible();
    placerElements(n,1,5);
}
```

Le constructeur de la classe Grille délègue la logique lourde à une méthode privée initialiser(). Cela permet de garder un constructeur propre et de pouvoir réinitialiser le jeu (reset) facilement sans créer une nouvelle grille.

la grille appelle ses méthodes de peuplement : placerCible() et placerElements(). Cela nous permet de séparer la structure (les cases) du contenu (les bonus/obstacles), ce qui rend le jeu modulable

Suivant

```java
public boolean seDeplacer(Cellule nouvelle) {
    Cellule actuelle = this.position;

    int dx = Math.abs(nouvelle.x - actuelle.x);
    int dy = Math.abs(nouvelle.y - actuelle.y);

    if (dx + dy != 1) {
        // pas une case adjacente
        System.out.println("Déplacement impossible (pas adjacent)");
        return false;

    }

    // element
    ElementInterface elementSurCase = nouvelle.getElement();

    if (elementSurCase != null) {
        int valeur = elementSurCase.getEnergie(); // getEnergie c'est l'interface

        if (valeur > 0) {
            System.out.println("Vous avez mangé un fruit : vous regagnez de l'énergie.");
            nouvelle.setElement(null);
        } else {
            System.out.println("Vous vous brûlez, cela vous fait perdre de l'énergie.");
        }

        this.energie = this.energie + valeur - 1;

} else {
    this.energie -= 1;
}


    // Déplacement effectif
    this.setPosition(nouvelle);

    this.historique.add(nouvelle);
```

NOTRE VERSION MINIMALE

On vérifie que le joueur ne saute pas de case
Ensuite on regarde si sur la case où il souhaite se déplacer il y a un élement si oui on affecte l'énergie. Chaque déplacement lui coute 1 énergie.

Le Chasseur ne bouge pas au hasard. Il scanne ses voisins (getVoisins) pour voir si sa cible est à portée. S'il voit la cible, il attaque. Sinon, il utilise un algorithme de recherche (getCaseVersCible) pour se rapprocher.

```java
public void jouerTourAuto(Grille grille, Bonhomme cibleBonhomme) {
    // Voisins du chasseur pour checker si il y a le bonhomme
    java.util.List<Cellule> voisins = grille.getVoisins(this.positio

    for (Cellule v : voisins) {
        if (v.equals(cibleBonhomme.getPosition())) {
            System.out.println("Bonhomme DETECTE ! Le chasseur attra
            this.seDeplacer(v);
            return;
        }
    }

    Cellule destination = grille.getCaseVersCible(this.position);
    this.seDeplacer(destination);
    System.out.println("Energie du chasseur : " + energie);
}
```

Suivant

Le patron de conception ADAPTATEUR

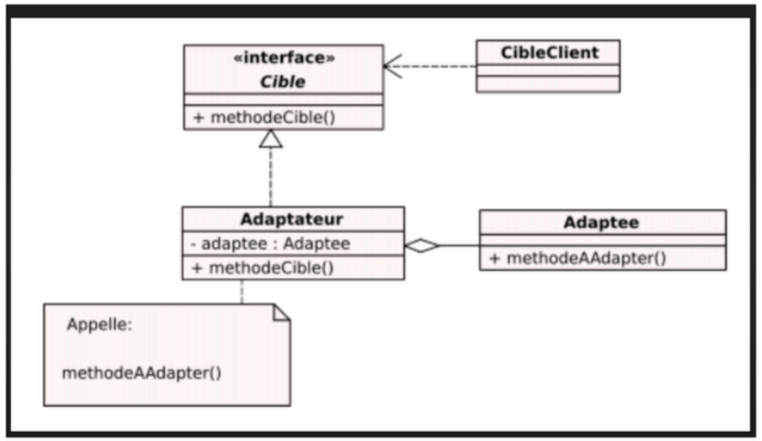


FIGURE 1 – Schéma du patron de conception adpatateur.

Objectif : Intégrer des bonus et des malus sans modifier le code existant du jeu (classes Cellule ou Joueur)

- Utilisation d'une interface commune : ElementInterface

Interface : fait le pont entre les méthodes spécifiques des classes et la méthode standard de Joueur qui gère l'énergie : getEnergie()

→ Ouvert à l'extension mais fermé à la modification

```java
package projet_final;

public class AdaptateurFeu implements ElementInterface {
  private Feu feu;

  public AdaptateurFeu(Feu feu) {
    this.feu = feu;
    }

  @Override
  public int getEnergie() {
    // TODO Auto-generated method stub
    return this.feu.energieMoins();
  }


}
```

Suivant

Le patron de conception STRATÉGIE

Objectif : créer de nouvelles stratégies de déplacement du bonhomme sans modifier les classes

Pour cela : Interface DeplacementStrategy qui permet de créer deux modes :
1) Stratégie Manuelle
2) Stratégie d'Exploration

Se déclenche lorsque l'énergie du bonhomme dépasse la valeur seuil.

→ Ce patron de conception permet de créer une multitude de stratégies sans modifier les classes concernées.

```java
public void jouerTour(Grille grille) {

    if (this.energie > highValue) {
        // Mode on choisit n'importe quelle case non explorée
      this.strategie = new StrategieAutre();
        System.out.println("-MODE EXPLO- Énergie haute (" + energie + ") : Exploration en cours...");
    } else {
        // Mode manuel avec les touches
      this.strategie = new StrategieManuelle();
        System.out.println("-MODE MANUEL- Énergie basse (" + energie + ") : À vous de jouer !");
    }

    // Là on demande la case à la stratégie
    Cellule destination = this.strategie.choisirCase(this, grille);

    //Déplacement
    if (destination != this.position) { // Si on a choisi une nouvelle case
        boolean success = this.seDeplacer(destination);
        if (success) {
            System.out.println("Déplacement vers (" + destination.x + "," + destination.y + ") réussi.");
        }
    } else {
        System.out.println("Vous restez sur place.");
    }
}
```

Suivant

PATTERN - OBESERVER

→ Les pattern nous permettent d'ajouter facilement des fonctionnalités, de rendre le code plus flexible sans pour autant modifier notre base initiale

```
@Override
  public void update(JeuEvent event) {
    switch(event.getType()) {

        case POSITION_CHANGE:
            verifierCollision();
            verifierCible();
            break;

        case ENERGY_CHANGE:
            Joueur j = (Joueur) event.getSource();
            if (!j.estVivant()) {
                System.out.println("Un joueur n'a plus d'énergie.
                partieEnCours = false;
            }
            break;

        case JEU_FINI:
            partieEnCours = false;
            break;
    }
    }
}
```

Le rôle de l'observer

- observer l'état du jeu notamment pour l'énergie, la position et la fin de partie
- Gestion par l'évènement
- Permet de séparer la logique de jeu et l'affichage

Suivant

PATTERN- FACTORY

```java
package projet_final;

public class ElementFactory {
    public static ElementInterface creerElement(int valeur) {
        if (valeur > 0) {
            Fruit fruit = new Fruit();
            return new AdaptateurFruit(fruit);
        }

        else {
            Feu feu = new Feu();
            return new AdaptateurFeu(feu);
        }
    }
}
```

Le rôle de l'ElementFactory

- centralisation de la création de nos éléments, permet de rendre le code beaucoup plus flexible
- réutilisation de nos adaptateurs
- possibilité d'évolution: gel, boost, etc

Suivant

Difficultés rencontrées

- Synchronisation de la fin du jeu avec le tour par tour
- La séparation entre les tâches de l'Observer et de la méthode jouerTour()
- Savoir où et comment implémenter la logique de déplacement
- Les rajouts des patrons de conception nous ont parfois apporté beaucoup de confusion et d'incohérences dans notre code
- Changement de consignes

Suivant

Pour aller plus loin

- Nous aurions pu étendre les prérogatives de l'Observer (centralisation de tout l'affichage du jeu)
- Conception d'une vraie interface graphique en utilisant SceneBuilder
- Enrichir le Game Play en créant de nouveaux éléments avec lesquels les joueurs peuvent interagir (comme des items à récolter qu'ils peuvent activer pendant leur tour)

Suivant

Merci pour votre attention